

Arduino IDE - MPU6050 V1.0

Date: 18/11/2025

Tags: [PEE](#), [Code](#), [Microcontroller](#), [ESP32](#), [Algorithms](#)

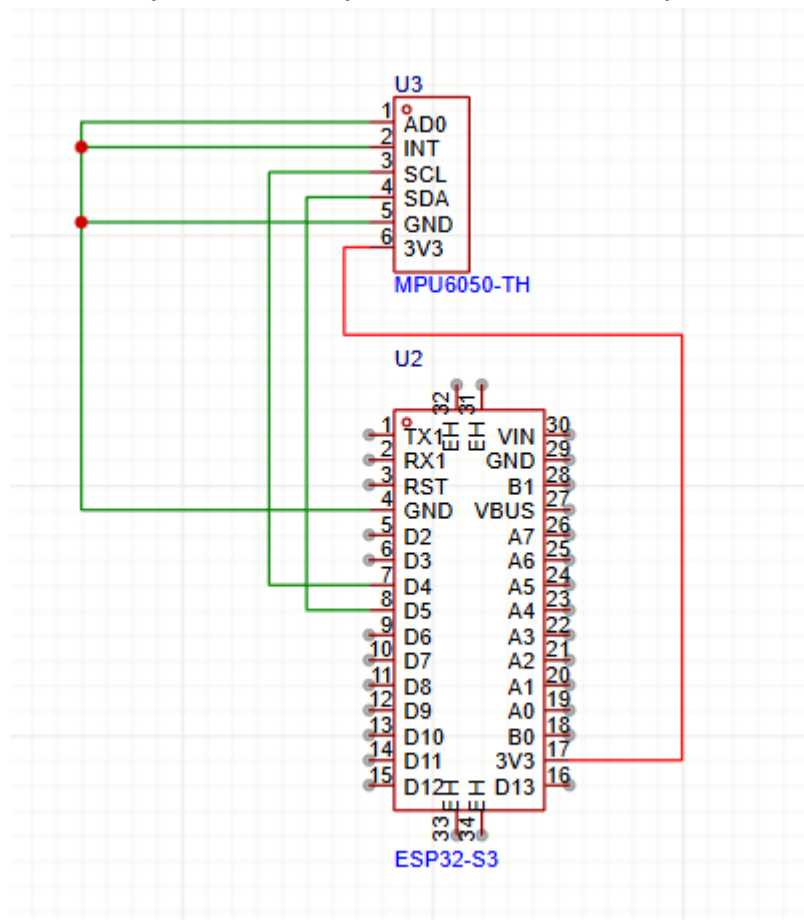
Description

This page main objective is documenting and explaining my progress with the code for the ESP32 to work only with our IMU, MPU6050.

The version 1.n is destined only for the MPU6050 implementation, the 2.n will be used for the integration with other modules from the localisation module and the final 3.0 will be implement with all the vest complements.

MPU 6050

The MPU6050 is a very simple IMU to connect. We really only need 4 pins connected to the MCU. The VCC pin, the GND pin, the SDA and SCL pin.



In the above image the AD0 and INT are connected but they can be connected to nothing.

Code

- For the code I will explained the various lines by snippets of code.

```
#include <Wire.h>
```

- This line of code imports the Wire.h library which is needed to communicate with devices that work with I2C protocol.

```

1  #define I2C_SDA 5
2  #define I2C_SCL 4

```

- This snippet of code defines the GPIO Pins on the ESP32 that act as data line (SDA) and clock line (SCL).

```
const int MPU_ADDR = 0x68;
```

- This line of code gives the address which the the MPU is in, this value was given by a previous code that scans the I2C address of each component ("I2C_Scanner").

```
const int h = 333;
```

- This line defines a constant **h** that is used as the sample time of the MPU, which sample time = 333 ms;

Setup code V1.0

```

1  void setup() {
2      Serial.begin(115200);
3      Wire.begin(I2C_SDA, I2C_SCL);
4      Wire.beginTransmission(MPU_ADDR);
5      Wire.write(0x6B); // Wake up MPU
6      Wire.write(0);
7      Wire.endTransmission(true);
8  }

```

- `Serial.begin(115200)` Sets the data rate in bits per second or baud for serial data transmission;
- `Wire.begin(I2C_SDA, I2C_SCL)` Initializes the Wire.h library and join the I2C bus as controller or a peripheral;
- `Wire.beginTransmission(MPU_ADDR)` Begins a transmission with the I2C peripheral device with the given address (0x68);
- `Wire.write(0x6B)` This function writes data from a peripheral device in response to a request from a controller device. The 0x6B address in the MPU-6050 datasheet is the PWR_MGMT_1 (Power Management 1). This register allows the user to configure the power mode and clock source. It also provides a bit for resetting the entire device, and a bit for disabling the temperature sensor.

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6B	107	DEVICE_RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		

- `Wire.endTransmission(true)` This function ends a transmission to a peripheral device that was begun by **beginTransmission()** and transmits the bytes that were queued by **write()**. This method accepts a boolean argument. If true, sends a stop message after transmission, if false , sends a restart message after transmission.

Loop Code V1.0

```

1  void loop(){
2      unsigned long currentTime = millis();
3

```

```

4     if ((currentTime - previousTime) >= h) {
5         // --- 1. LER OS DADOS RAW (Inteiros) ---
6         Wire.beginTransmission(MPU_ADDR);
7         Wire.write(0x3B);
8         Wire.endTransmission(false);
9         Wire.requestFrom(MPU_ADDR, 14, true);
10
11         AcX_raw = Wire.read() << 8 | Wire.read();
12         AcY_raw = Wire.read() << 8 | Wire.read();
13         AcZ_raw = Wire.read() << 8 | Wire.read();
14         Tmp_raw = Wire.read() << 8 | Wire.read();
15         GyX_raw = Wire.read() << 8 | Wire.read();
16         GyY_raw = Wire.read() << 8 | Wire.read();
17         GyZ_raw = Wire.read() << 8 | Wire.read();
18
19         Serial.print("AcX = "); Serial.print(ax, 4); // m/s²
20         Serial.print(" AcY = "); Serial.print(ay, 4); // m/s²
21         Serial.print(" AcZ = "); Serial.print(az, 4); // m/s² (Deve ser ~9.81)
22         Serial.print(" GyX = "); Serial.print(gx, 4); // °/s
23         Serial.print(" GyY = "); Serial.print(gy, 4); // °/s
24         Serial.print(" GyZ = "); Serial.println(gz, 4); // °/s
25
26         previousTime = currentTime;
27     }
28 }

```

- `unsigned long currentTime = millis()` The function **millis()** return the number of milliseconds passed since the MCU began running the current program. This number will overflow after approximately 50 days.
- `if((currentTime - previousTime) >= h)` This condition makes sure that in each iteration is between the sample time.
- `Wire.write(0x3B)` This function writes data from a peripheral device in response to a request from a controller device. The 0x3B address in the MPU-6050 is the first address for "Accelerometer Measurements" (ACCEL_XOUT [15:8]).
- `Wire.requestFrom(MPU_ADDR, 14, true);` This function is used by the controller device to request bytes from a peripheral device. In this case the MCU will request 14 bytes from the MPU-6050 address. But with the previous line this will ask the MPU-6050 will ask bytes of data starting at register 0x3B. 0x3B till 0x48 are the addresses of the MPU-6050 that has the Accelerometer, Gyroscope and Temperature measurements.
- `AcX_raw = Wire.read() << 8 | Wire.read()` The MPU-6050 sends data in two parts : a "High Byte" and "Low Byte", this line shifts the high byte 8 spots to the left and combines it with the low byte using OR (|). This will reconstruct the full 16-bit integer.
- `Serial.print("AcX = "); Serial.print(AcX_raw, 4)` Prints the raw data.
- `previousTime = currentTime` With this line we can assure the boolean condition.

Mistakes

- Our microcontroller unit (MCU) is a ESP32-S3-WROOM-1, but has it's integrated in an unofficial board. This means that our MCU is not from *ESPRESSIF*, how did I see this? By plugging the MCU

instead of one COM port connected, it appears two of them (e.g COM10, ENCHANTED-SERIAL-USB-CH343). And my mistake was uploading the code to the COM10 and not the other one. In this types of board the COM with CH343 is who does the bridge between computer's USB port to a serial UART interface, allowing communication with microcontrollers.

- Using a library that works only with a specific component to cut work. Initially I downloaded the ADAFRUIT MPU-6050 to make it easier. Unfortunately the library only works with the MPU 6050 from ADAFRUIT.

Bugs and Fixes

I2C Error

So as our IMU communicates via I2C protocol, I needed to connected the SCL/SCK (clock signal) pin and the SDA/SDI (send and receive data) pin from the IMU to the ones from the MCU. For this I used a code from *Random Nerd Tutorials*(<https://randomnerdtutorials.com/esp32-i2c-communication-arduino-ide/>) named as "I2C Scanner", which fetches the I2C address that our IMU is in. The bug was that it failed and why it failed?

Firstly I searched about the SDA and SCL pins of our MCU breakout board and in *Random Nerd Tutorials* I found a page about the pinout (<https://randomnerdtutorials.com/esp32-s3-devkitc-pinout-guide/>). Connecting the pins with the information that I obtained in this website and it failed. So how did I fix it? Well, I searched about I2C in ESP32 board and end up with the information that ESP32 board I2C pins are configurable with this simple snippet of code:

```
1  #define I2C_SDA 5
2  #define I2C_SCL 4
3
4  void setup() {
5
6      Serial.begin(115200);
7      Wire.begin(I2C_SDA,I2C_SCL); // This is the line that matters the most
8      delay(100);
9      Serial.println("\nI2C Scanner");
10 }
```

And this is the whole code for the I2C Scanner that was exported from Random Nerd Tutorials:

I2C_Scanner

```
1  /*****
2      Rui Santos
3      Complete project details at https://randomnerdtutorials.com
4      *****/
5
6  #include <Wire.h>
7
8  #define I2C_SDA 5
9  #define I2C_SCL 4
10
11 void setup() {
12
```

```

13   Serial.begin(115200);
14   Wire.begin(I2C_SDA,I2C_SCL);
15   delay(100);
16   Serial.println("\nI2C Scanner");
17 }
18
19 void loop() {
20
21   byte error, address;
22   int nDevices;
23   Serial.println("Scanning...");
24   nDevices = 0;
25   for(address = 1; address < 127; address++ ) {
26     Wire.beginTransmission(address);
27     error = Wire.endTransmission();
28     if (error == 0) {
29       Serial.print("I2C device found at address 0x");
30       if (address<16) {
31         Serial.print("0");
32       }
33       Serial.println(address,HEX);
34       nDevices++;
35     }
36     else if (error==4) {
37       Serial.print("Unknow error at address 0x");
38       if (address<16) {
39         Serial.print("0");
40       }
41       Serial.println(address,HEX);
42     }
43   }
44   if (nDevices == 0) {
45     Serial.println("No I2C devices found\n");
46   }
47   else {
48     Serial.println("done\n");
49   }
50   delay(5000);
51 }

```

Datasheet

<https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>